

DSA Revision Notes: Arrays and Stacks

Last-minute revision guide — built for quick reading the night before the exam. Every operation follows the same 7-part structure: Concept → Algorithm → Step-by-Step → Dry Run → Memory Trick → Common Mistakes → Viva Questions.

ARRAYS

Quick refresher: An array is a collection of elements stored in **contiguous (side-by-side) memory locations**, all of the **same data type**, accessed using an **index** (position number, usually starting from 0).

1. Traversal

1. Concept

- **What it does:** Visits every element of the array, one by one, from the first to the last position.
- **Why it is needed:** It is the foundation of almost every array operation — you cannot search, print, or process data without visiting each element at least once.
- **When it is used:** Printing all elements, calculating sum/average, finding maximum/minimum, or applying any operation to every item.

Case	Time Complexity	Why
Best	$O(N)$	Every element must still be visited once
Average	$O(N)$	Same — no shortcuts possible
Worst	$O(N)$	Same — traversal always visits all N elements

Note: Traversal has no "best case shortcut" — it always takes time proportional to the number of elements (N).

2. Algorithm

```
ALGORITHM: TraverseArray
INPUT: LinearArray, NumberOfElements
1. Start
2. Set CurrentIndex to 0
3. Repeat steps 4 to 6 while CurrentIndex is less than NumberOfElements
4.     Access the element at position CurrentIndex in LinearArray
5.     Process or display the accessed element
6.     Increase CurrentIndex by 1
7. End Repeat
8. Stop
```

3. Step-by-Step Explanation

1. **Start** — this simply marks the beginning of the algorithm.
2. We set a counter called `CurrentIndex` to `0` because arrays usually start counting positions from 0, not 1.
3. We use a loop (a repeating block) that keeps running **as long as** `CurrentIndex` is smaller than the total number of elements. This ensures we stop exactly when we run out of elements.
4. Inside the loop, we look at (access) whatever value is sitting at the current position.
5. We do something useful with that value — print it, add it to a total, compare it, etc.
6. We move to the next position by increasing `CurrentIndex` by 1. Without this step, the loop would run forever.
7. Once `CurrentIndex` becomes equal to `NumberOfElements`, the condition in step 3 becomes false, and the loop stops.
8. **Stop** — the algorithm ends.

4. Dry Run

Array: `[10, 20, 30, 40]`, `NumberOfElements = 4`

CurrentIndex	Condition (Index < 4)	Element Accessed	Array State
0	True	10	[10, 20, 30, 40]
1	True	20	[10, 20, 30, 40]
2	True	30	[10, 20, 30, 40]
3	True	40	[10, 20, 30, 40]
4	False → Loop Ends	—	[10, 20, 30, 40]

Output: `10 20 30 40`

5. Memory Trick

"Start at Zero → Check → Visit → Move → Repeat"

6. Common Mistakes

- Starting the index at `1` instead of `0`, causing the first element to be skipped.
- Using `<=` instead of `<` in the condition, which causes an **out-of-bounds error** (accessing a position that doesn't exist).
- Forgetting to increase the index inside the loop, causing an **infinite loop**.
- Not stopping at `NumberOfElements - 1` (the last valid index).

7. Viva Questions

1. **Q: Why does array indexing start at 0?** A: Because the index represents the "offset" (distance) from the starting memory address, and the first element has zero offset.
 2. **Q: What is the time complexity of traversal, and why can't it be improved?** A: $O(N)$, because every element must be visited at least once — there's no way to skip elements while still checking all of them.
 3. **Q: What error occurs if the loop condition uses `<=` instead of `<`?** A: An out-of-bounds (index-out-of-range) error, because it tries to access one position beyond the last valid index.
 4. **Q: Is traversal a searching technique?** A: No — traversal simply visits every element; searching involves comparing elements against a target value.
-

2. Insertion

1. Concept

- **What it does:** Adds a new element into the array at a specified position, shifting existing elements to make room.
- **Why it is needed:** Real-world data changes — new records, new scores, new entries need to be added without destroying existing data.
- **When it is used:** Adding a new student's marks, inserting a new item into a sorted list, adding a node's data in array-based implementations.

Case	Time Complexity	Why
Best	$O(1)$	Inserting at the end (no shifting needed)
Average	$O(N)$	On average, half the elements must shift

Case	Time Complexity	Why
Worst	$O(N)$	Inserting at the beginning — all elements shift right

Important: The array must have empty space available; otherwise, insertion is not possible (unless resizing is allowed).

2. Algorithm

```

ALGORITHM: InsertElement
INPUT: LinearArray, NumberOfElements, PositionToInsert, ValueToInsert
1. Start
2. If NumberOfElements equals MaximumCapacity, then display "Array is Full"
   and Stop
3. Set CurrentIndex to NumberOfElements
4. Repeat steps 5 to 6 while CurrentIndex is greater than PositionToInsert
5.     Copy the element at (CurrentIndex - 1) to CurrentIndex
6.     Decrease CurrentIndex by 1
7. End Repeat
8. Place ValueToInsert at PositionToInsert
9. Increase NumberOfElements by 1
10. Stop

```

3. Step-by-Step Explanation

1. **Start** the process.
2. First, check if the array is already full. If it is, there is no room to insert, so we stop immediately.
3. We begin from the **last used position** (`NumberOfElements`), because we need to shift elements from the end, not the beginning — shifting from the front would overwrite data.
4. We keep shifting elements **as long as** our current position is still ahead of the position where we want to insert.
5. We copy the element that is just before the current position into the current position — this is the actual "shifting right" action.
6. We move one step backward (toward the front) so the next shift happens correctly.
7. This loop stops once we reach the exact position where the new value should go.
8. Now that a "gap" has been created at `PositionToInsert` , we place the new value there.
9. Since we added one new element, we increase the total count of elements.
10. **Stop**.

4. Dry Run

Array: [10, 20, 30, _] (capacity 4, NumberOfElements = 3), Insert 15 at
PositionToInsert = 1

Step	Action	Array State
Initial	—	[10, 20, 30, _]
CurrentIndex = 3	Copy element at index 2 (30) to index 3	[10, 20, 30, 30]
CurrentIndex = 2	Copy element at index 1 (20) to index 2	[10, 20, 20, 30]
CurrentIndex = 1	Loop stops (1 is not > 1)	[10, 20, 20, 30]
Place value	Place 15 at index 1	[10, 15, 20, 30]
Update count	NumberOfElements = 4	[10, 15, 20, 30]

Final Array: [10, 15, 20, 30]

5. Memory Trick

"Check Space → Shift Right from End → Drop the New Value → Count Up"

6. Common Mistakes

- Shifting elements **from the front instead of the back**, which overwrites data instead of creating a gap.
- Forgetting to check if the array is full before inserting.
- Forgetting to increase `NumberOfElements` after insertion.
- Off-by-one errors in the loop condition, causing either a skipped element or an overwritten one.

7. Viva Questions

1. **Q: Why do we shift elements from the end rather than from the beginning?** A: Shifting from the beginning would overwrite values before they are copied forward; shifting from the end preserves all data correctly.
 2. **Q: What happens if you try to insert into a full array?** A: It results in an overflow condition — the operation should be blocked with an "Array is Full" message.
 3. **Q: Why is inserting at the beginning the worst case?** A: Because every single existing element must shift one position to the right, taking $O(N)$ time.
 4. **Q: Can insertion be done in $O(1)$ time in a normal array?** A: Only when inserting at the end **and** space is available — no shifting is required in that case.
-

3. Deletion

1. Concept

- **What it does:** Removes an element from a specified position and shifts the remaining elements to fill the gap.
- **Why it is needed:** Data often needs to be removed — a cancelled order, a graduated student's record, an expired entry.
- **When it is used:** Removing duplicate values, deleting an invalid entry, removing an item from a list-based structure.

Case	Time Complexity	Why
Best	$O(1)$	Deleting the last element (no shifting needed)
Average	$O(N)$	On average, half the elements shift left
Worst	$O(N)$	Deleting the first element — all remaining elements shift left

2. Algorithm

```
ALGORITHM: DeleteElement
INPUT: LinearArray, NumberOfElements, PositionToDelete
1. Start
2. If NumberOfElements equals 0, then display "Array is Empty" and Stop
3. If PositionToDelete is not between 0 and (NumberOfElements - 1), then
   display "Invalid Position" and Stop
4. Set CurrentIndex to PositionToDelete
5. Repeat steps 6 to 7 while CurrentIndex is less than (NumberOfElements - 1)
6.     Copy the element at (CurrentIndex + 1) to CurrentIndex
7.     Increase CurrentIndex by 1
8. End Repeat
9. Decrease NumberOfElements by 1
10. Stop
```

3. Step-by-Step Explanation

1. **Start** the process.
2. If there are no elements at all, there's nothing to delete, so we stop with a message.
3. We check that the position given actually exists in the array; otherwise the operation is invalid.
4. We begin at the position we want to delete, because this is where the "gap" will first appear.
5. We keep shifting **as long as** there are elements to the right of the current position.

6. We copy the value from the **next** position into the **current** position — this is the "shifting left" action that closes the gap.
7. We move one step forward to continue closing the gap for the next pair of positions.
8. The loop naturally stops once we reach the last element, since there's nothing left to shift.
9. Since one element has been removed, we decrease the total element count by 1.
10. **Stop.**

4. Dry Run

Array: [10, 15, 20, 30] , NumberOfElements = 4 , Delete element at PositionToDelete = 1 (value 15)

Step	Action	Array State
Initial	—	[10, 15, 20, 30]
CurrentIndex = 1	Copy element at index 2 (20) to index 1	[10, 20, 20, 30]
CurrentIndex = 2	Copy element at index 3 (30) to index 2	[10, 20, 30, 30]
CurrentIndex = 3	Loop stops (3 is not < 3)	[10, 20, 30, 30]
Update count	NumberOfElements = 3	[10, 20, 30] (last slot ignored)

Final Array: [10, 20, 30]

5. Memory Trick

"Check Empty → Shift Left from Gap → Count Down"

6. Common Mistakes

- Shifting elements **from the end instead of the position of deletion**, missing the gap-closing effect.
- Forgetting to reduce `NumberOfElements` , leaving a "ghost" duplicate value logically in the array.
- Not validating the position, leading to deleting a non-existent element.
- Confusing deletion with simply setting a value to zero or null (that isn't true deletion — the gap must be closed).

7. Viva Questions

1. **Q: Why does deletion shift elements to the left?** A: To close the gap left behind by the removed element, keeping all elements contiguous.
 2. **Q: What is the best case for deletion, and why?** A: Deleting the last element — $O(1)$, because no elements need to shift.
 3. **Q: What happens to the last duplicate value after deletion?** A: It becomes logically unused because `NumberOfElements` is decreased, even though the physical memory slot may still hold the old value.
 4. **Q: Why is deleting the first element the worst case?** A: Because every remaining element must shift one position left, taking $O(N)$ time.
-

4. Linear Search

1. Concept

- **What it does:** Searches for a target value by checking each element **one by one**, starting from the first, until it is found or the array ends.
- **Why it is needed:** It is the simplest way to search when data is **unsorted** or when the array is small.
- **When it is used:** Searching in unsorted arrays, or arrays too small to justify more complex searching.

Case	Time Complexity	Why
Best	$O(1)$	The target is the very first element
Average	$O(N)$	On average, the target is found halfway through
Worst	$O(N)$	The target is the last element, or not present at all

2. Algorithm

```
ALGORITHM: LinearSearch
INPUT: LinearArray, NumberOfElements, TargetValue
1. Start
2. Set CurrentIndex to 0
3. Repeat steps 4 to 6 while CurrentIndex is less than NumberOfElements
4.     If the element at CurrentIndex equals TargetValue, then display "Found
at Position CurrentIndex" and Stop
5.     Increase CurrentIndex by 1
6. End Repeat
```


7. Display "Value Not Found"
8. Stop

3. Step-by-Step Explanation

1. **Start** the process.
2. We begin checking from position 0, the very first element.
3. We keep looping **as long as** we haven't gone past the last element.
4. At every position, we compare the current element with the value we are searching for. If they match, we immediately announce success and stop — there's no need to check further.
5. If they don't match, we move on to check the next position.
6. This loop continues position by position.
7. If the loop finishes completely without finding a match, we know the value doesn't exist in the array.
8. **Stop**.

4. Dry Run

Array: [5, 8, 2, 9, 1] , Search for TargetValue = 9

CurrentIndex	Element	Match with 9?	Action
0	5	No	Move to next
1	8	No	Move to next
2	2	No	Move to next
3	9	Yes	Display "Found at Position 3"

Output: Found at Position 3

5. Memory Trick

"Search → Compare → Found? → Move → Repeat"

6. Common Mistakes

- Not stopping the search immediately after finding the target (continuing to loop unnecessarily).
- Forgetting to handle the "not found" case after the loop ends.
- Using linear search on large sorted arrays when binary search would be far more efficient.

- Comparing the wrong variable (e.g., comparing `CurrentIndex` with `TargetValue` instead of the element itself).

7. Viva Questions

1. **Q: When is linear search preferred over binary search?** A: When the array is unsorted, or when the array is very small (sorting overhead isn't worth it).
 2. **Q: What is the worst-case time complexity of linear search?** A: $O(N)$, occurring when the target is at the last position or absent entirely.
 3. **Q: Does linear search require a sorted array?** A: No — it works correctly on both sorted and unsorted arrays.
 4. **Q: What is the best-case scenario for linear search?** A: When the target value is found at the very first position — $O(1)$.
-

5. Binary Search

1. Concept

- **What it does:** Searches for a target value in a **sorted** array by repeatedly dividing the search range in half.
- **Why it is needed:** It is dramatically faster than linear search for large, sorted datasets.
- **When it is used:** Searching in sorted arrays — dictionaries, phonebooks, sorted database indexes.

Case	Time Complexity	Why
Best	$O(1)$	The target is exactly at the middle on the first check
Average	$O(\log N)$	Search range is halved with each comparison
Worst	$O(\log N)$	Search continues until only one element remains

Golden Rule: Binary search **only works on sorted arrays**. Using it on an unsorted array gives incorrect results.

2. Algorithm

```
ALGORITHM: BinarySearch
INPUT: SortedArray, NumberOfElements, TargetValue
1. Start
2. Set LowerBound to 0
```

```
3. Set UpperBound to (NumberOfElements - 1)
4. Repeat steps 5 to 10 while LowerBound is less than or equal to UpperBound
5.     Set MiddleIndex to (LowerBound + UpperBound) divided by 2
6.     If the element at MiddleIndex equals TargetValue, then display "Found
at Position MiddleIndex" and Stop
7.     If the element at MiddleIndex is less than TargetValue, then set
LowerBound to (MiddleIndex + 1)
8.     Otherwise, set UpperBound to (MiddleIndex - 1)
9. End Repeat
10. Display "Value Not Found"
11. Stop
```

3. Step-by-Step Explanation

1. **Start** the process.
2. We set the lower boundary of our search range to the very first index.
3. We set the upper boundary of our search range to the very last index.
4. We keep searching **as long as** the lower boundary hasn't crossed past the upper boundary — meaning there's still a valid range to check.
5. We calculate the position exactly in the **middle** of the current range.
6. We check if the middle element is our target. If yes, we're done — no need to search further.
7. If the middle element is **smaller** than the target, our target must be in the **right half**, so we move the lower boundary just past the middle.
8. Otherwise (if the middle element is **larger**), our target must be in the **left half**, so we move the upper boundary just before the middle.
9. This process repeats, cutting the search range roughly in half every time.
10. If the range becomes invalid (lower boundary crosses upper boundary) without finding the target, it means the value isn't in the array.
11. **Stop**.

4. Dry Run

Sorted Array: [2, 5, 8, 12, 16, 23, 38] (indices 0–6), Search for `TargetValue = 23`

Step	LowerBound	UpperBound	MiddleIndex	Middle Value	Comparison
1	0	6	3	12	$12 < 23 \rightarrow$ search right half
2	4	6	5	23	Match found!

Output: Found at Position 5

5. Memory Trick

"Sorted? → Find Middle → Too Small Go Right, Too Big Go Left → Repeat"

6. Common Mistakes

- Using binary search on an **unsorted** array (a very common exam trap).
- Incorrectly computing the middle index, e.g., forgetting integer division.
- Setting the new boundary to `MiddleIndex` instead of `MiddleIndex + 1` or `MiddleIndex - 1`, causing an **infinite loop**.
- Forgetting the "not found" case when `LowerBound` crosses `UpperBound`.

7. Viva Questions

1. **Q: Why must the array be sorted for binary search?** A: Because the algorithm eliminates half the array based on comparison, which is only valid if elements are in order.
 2. **Q: Why is binary search's time complexity $O(\log N)$?** A: Because the search range is halved with every comparison, so it takes about $\log_2(N)$ steps to reduce the range to one element.
 3. **Q: What happens if the array has duplicate values?** A: Standard binary search will find **any** matching occurrence, not necessarily the first or last one, unless modified.
 4. **Q: Compare linear search and binary search in terms of speed.** A: Binary search ($O(\log N)$) is much faster than linear search ($O(N)$) for large sorted arrays, but requires the data to be sorted first.
 5. **Q: Can binary search be applied to a linked list efficiently?** A: No — binary search needs direct (random) access to the middle element, which linked lists don't support efficiently.
-

6. Updating

1. Concept

- **What it does:** Changes the value stored at a specific, already-known index to a new value.
- **Why it is needed:** Data changes over time — a price change, a corrected score, an updated status.
- **When it is used:** Editing a record, correcting an entry, refreshing a value (e.g., updating a live score).

Case	Time Complexity	Why
Best	$O(1)$	Direct access using the index — no traversal or shifting needed
Average	$O(1)$	Same — index-based access is always direct
Worst	$O(1)$	Same — arrays support constant-time direct access

Key Insight: Updating is the **fastest** array operation because arrays support **random access** — you can jump straight to any index.

2. Algorithm

```
ALGORITHM: UpdateElement
INPUT: LinearArray, NumberOfElements, PositionToUpdate, NewValue
1. Start
2. If PositionToUpdate is not between 0 and (NumberOfElements - 1), then
   display "Invalid Position" and Stop
3. Set the element at PositionToUpdate to NewValue
4. Display "Update Successful"
5. Stop
```

3. Step-by-Step Explanation

- Start** the process.
- We first check that the given position actually exists within the current array bounds.
Updating an invalid position would corrupt memory or cause an error.
- We directly overwrite the old value at that position with the new value — this is possible instantly because arrays allow direct jumping to any index.
- We confirm the update was successful.
- Stop.**

4. Dry Run

Array: [10, 20, 30, 40] , Update position 2 to value 99

Step	Action	Array State
Initial	—	[10, 20, 30, 40]
Validate	Position 2 is valid (0 to 3)	[10, 20, 30, 40]
Update	Set index 2 to 99	[10, 20, 99, 40]

Final Array: [10, 20, 99, 40]

5. Memory Trick

"Check Position → Overwrite Directly → Done"

6. Common Mistakes

- Forgetting to validate the position before updating, risking an out-of-bounds error.
- Confusing updating with insertion (updating does **not** shift any elements).
- Assuming updating takes $O(N)$ time like insertion or deletion — it does not.
- Updating the wrong index due to off-by-one confusion.

7. Viva Questions

1. **Q: Why is updating an array element $O(1)$?** A: Because arrays support random (direct) access — the address of any index can be calculated instantly using the base address and index.
 2. **Q: Does updating change the size of the array?** A: No, the number of elements stays exactly the same.
 3. **Q: How is updating different from insertion?** A: Updating overwrites an existing value at a known position; insertion adds a brand-new value and shifts other elements.
 4. **Q: What is "random access" in the context of arrays?** A: The ability to access any element directly using its index, without needing to traverse from the beginning.
-

7. Merging

1. Concept

- **What it does:** Combines two arrays into a single array, typically preserving order (and often merging two sorted arrays into one sorted array).
- **Why it is needed:** Useful for combining datasets — merging two sorted lists is also the core step of **Merge Sort**.
- **When it is used:** Combining two sorted lists of scores, merging results from two data sources, the merge step of merge sort.

Case	Time Complexity	Why
Best	$O(N + M)$	Every element from both arrays must be copied once
Average	$O(N + M)$	Same reasoning — no shortcuts

Case	Time Complexity	Why
Worst	$O(N + M)$	Same — N and M are sizes of the two arrays

2. Algorithm

ALGORITHM: MergeTwoSortedArrays

INPUT: FirstArray, FirstArraySize, SecondArray, SecondArraySize

1. Start
2. Create MergedArray of size (FirstArraySize + SecondArraySize)
3. Set FirstIndex to 0, SecondIndex to 0, MergedIndex to 0
4. Repeat steps 5 to 8 while FirstIndex is less than FirstArraySize AND SecondIndex is less than SecondArraySize
 5. If element at FirstIndex in FirstArray is less than or equal to element at SecondIndex in SecondArray
 6. Copy element at FirstIndex from FirstArray to MergedArray at MergedIndex, then increase FirstIndex by 1
 7. Otherwise
 8. Copy element at SecondIndex from SecondArray to MergedArray at MergedIndex, then increase SecondIndex by 1
9. End Repeat (increase MergedIndex by 1 after every copy)
10. Copy any remaining elements from FirstArray (if any) into MergedArray
11. Copy any remaining elements from SecondArray (if any) into MergedArray
12. Stop

3. Step-by-Step Explanation

1. **Start** the process.
2. We prepare a new array large enough to hold **all** elements from both input arrays.
3. We set up three pointers: one for tracking our position in the first array, one for the second array, and one for where we're currently writing into the merged array.
4. We keep comparing elements **as long as** both arrays still have unprocessed elements.
5. We compare the current elements from both arrays.
6. If the first array's current element is smaller or equal, we copy it into the merged array — this keeps the result in sorted order — and move forward in the first array.
7. Otherwise, the second array's element must be smaller.
8. We copy that element instead, and move forward in the second array.
9. After every copy, we move forward in the merged array too, ready for the next value.
10. Once one array runs out of elements, the loop stops. Any elements still remaining in the first array (already sorted) are simply copied over in order.
11. Similarly, any remaining elements in the second array are copied over.

12. **Stop** — the merged array is now complete and sorted.

4. Dry Run

FirstArray: [2, 5, 8], SecondArray: [1, 6, 9]

Step	FirstIndex	SecondIndex	Comparison	Copied Value	MergedArray So Far
1	0	0	2 vs 1 → 1 smaller	1	[1]
2	0	1	2 vs 6 → 2 smaller	2	[1, 2]
3	1	1	5 vs 6 → 5 smaller	5	[1, 2, 5]
4	2	1	8 vs 6 → 6 smaller	6	[1, 2, 5, 6]
5	2	2	8 vs 9 → 8 smaller	8	[1, 2, 5, 6, 8]
6	3	2	FirstArray exhausted	copy remaining (9)	[1, 2, 5, 6, 8, 9]

Final Merged Array: [1, 2, 5, 6, 8, 9]

5. Memory Trick

"Compare Fronts → Copy Smaller → Slide That Pointer → Repeat → Dump the Leftovers"

6. Common Mistakes

- Forgetting to copy the **leftover elements** after one array is exhausted.
- Using `<` instead of `<=` in the comparison, which can disturb stability when equal values exist.
- Not creating a merged array of the correct total size beforehand.
- Confusing merging with simple concatenation (joining without maintaining sorted order).

7. Viva Questions

1. **Q: What is the time complexity of merging two sorted arrays of size N and M?** A: $O(N + M)$, because every element from both arrays is visited and copied exactly once.

2. **Q: Where is the merge operation used in a well-known algorithm?** A: It is the core combining step of **Merge Sort**.
3. **Q: Does merging require the input arrays to be sorted?** A: For the standard efficient merge algorithm, yes — both input arrays must already be sorted.
4. **Q: What happens if one array becomes empty before the other during merging?** A: The remaining elements of the non-empty array are copied directly into the merged array, since they are already in order.
-

8. Reversing

1. Concept

- **What it does:** Rearranges the elements of an array so that the first element becomes the last, and vice versa.
- **Why it is needed:** Useful for reversing order of display, checking palindromes, or reversing the effect of an earlier operation.
- **When it is used:** Reversing a sentence stored as characters, reversing a sequence of moves, palindrome checking.

Case	Time Complexity	Why
Best	$O(N)$	Every pair of elements must be swapped once
Average	$O(N)$	Same reasoning
Worst	$O(N)$	Same — proportional to array size (roughly $N/2$ swaps)

2. Algorithm

```
ALGORITHM: ReverseArray
INPUT: LinearArray, NumberOfElements
1. Start
2. Set StartIndex to 0
3. Set EndIndex to (NumberOfElements - 1)
4. Repeat steps 5 to 7 while StartIndex is less than EndIndex
5.     Swap the element at StartIndex with the element at EndIndex
6.     Increase StartIndex by 1
7.     Decrease EndIndex by 1
8. End Repeat
9. Stop
```

3. Step-by-Step Explanation

1. **Start** the process.
2. We place a pointer at the very beginning of the array.
3. We place another pointer at the very end of the array.
4. We continue swapping **as long as** the start pointer hasn't crossed or met the end pointer.
5. We exchange (swap) the values at these two pointer positions — the front value moves to the back, and the back value moves to the front.
6. We move the start pointer one step forward, toward the middle.
7. We move the end pointer one step backward, also toward the middle.
8. This continues until both pointers meet (or cross) in the middle, at which point every pair has been swapped.
9. **Stop** — the array is now fully reversed.

4. Dry Run

Array: [1, 2, 3, 4, 5]

Step	StartIndex	EndIndex	Action	Array State
1	0	4	Swap index 0 and 4	[5, 2, 3, 4, 1]
2	1	3	Swap index 1 and 3	[5, 4, 3, 2, 1]
3	2	2	Loop stops (StartIndex not < EndIndex)	[5, 4, 3, 2, 1]

Final Array: [5, 4, 3, 2, 1]

5. Memory Trick

"Two Pointers, Ends Toward Middle → Swap → Step In → Repeat"

6. Common Mistakes

- Using `<=` instead of `<` in the loop condition, causing the middle element (in an odd-length array) to be swapped with itself unnecessarily (not wrong, but wasteful) or, worse, causing pointers to cross incorrectly.
- Forgetting to move **both** pointers, leading to an infinite loop.
- Trying to swap without using a temporary variable (in manual/paper algorithms), causing data loss.

- Reversing only part of the array due to incorrect `EndIndex` initialization.

7. Viva Questions

1. **Q: Why does the loop run only $N/2$ times instead of N times?** A: Because each swap fixes two elements (front and back) at once, so only half the array needs to be visited.
 2. **Q: What happens to the middle element in an array with an odd number of elements?** A: It stays in place since it never gets paired with another distinct index.
 3. **Q: Can reversing be done without using extra array space?** A: Yes — the two-pointer swap method reverses the array **in-place**, using no extra array.
 4. **Q: What is the time complexity of reversing an array?** A: $O(N)$, since roughly $N/2$ swaps are performed, and constants are dropped in Big-O notation.
-

STACKS

Quick refresher: A stack is a linear data structure that follows the **LIFO** principle — **Last In, First Out**. Think of a stack of plates: you can only add or remove from the **top**.

9. Push

1. Concept

- **What it does:** Adds a new element to the **top** of the stack.
- **Why it is needed:** It is the only way to add data into a stack, always maintaining the LIFO order.
- **When it is used:** Adding a function call to the call stack, adding an undo action, adding a character while checking balanced brackets.

Case	Time Complexity	Why
Best	$O(1)$	Direct insertion at the top — no shifting needed
Average	$O(1)$	Same — always a direct operation
Worst	$O(1)$	Same — push never depends on stack size

2. Algorithm

```
ALGORITHM: PushElement
INPUT: StackArray, TopPointer, MaximumSize, ValueToPush
1. Start
2. If TopPointer equals (MaximumSize - 1), then display "Stack Overflow" and Stop
3. Increase TopPointer by 1
4. Set the element at TopPointer in StackArray to ValueToPush
5. Display "Push Successful"
6. Stop
```

3. Step-by-Step Explanation

- 1. **Start** the process.
- 2. We first check if the stack is already full (this condition is called **Stack Overflow**). If so, we cannot push, and we stop.
- 3. We increase the `TopPointer` by one position — this reserves the next free slot for our new value.
- 4. We place the new value into the stack at this new top position.
- 5. We confirm the operation succeeded.
- 6. **Stop**.

4. Dry Run

StackArray (capacity 5): `[10, 20, _, _, _]`, `TopPointer = 1`, Push value `30`

Step	Action	TopPointer	Stack State
Initial	—	1	<code>[10, 20, _, _, _]</code>
Check Overflow	<code>TopPointer (1) ≠ 4 → proceed</code>	1	<code>[10, 20, _, _, _]</code>
Increase TopPointer	<code>TopPointer becomes 2</code>	2	<code>[10, 20, _, _, _]</code>
Place Value	<code>Set index 2 to 30</code>	2	<code>[10, 20, 30, _, _]</code>

Final Stack: `[10, 20, 30]`, `TopPointer = 2`

5. Memory Trick

| **"Check Full → Step Up → Drop It On Top"**

6. Common Mistakes

- Increasing `TopPointer` **after** placing the value instead of **before**, causing the value to overwrite an existing element.
- Forgetting to check for Stack Overflow before pushing.
- Confusing `TopPointer` starting value — some implementations start at `-1` (empty stack) versus `0` (first slot used); mixing these up causes errors.
- Not updating `TopPointer` at all, so the stack always overwrites the same slot.

7. Viva Questions

1. **Q: What does "Stack Overflow" mean?** A: It means an attempt was made to push an element into a stack that is already full.
 2. **Q: Why is push an $O(1)$ operation?** A: Because it only involves incrementing a pointer and placing a value — no traversal or shifting is needed.
 3. **Q: What is the initial value of `TopPointer` for an empty stack?** A: Typically `-1`, indicating no elements are present yet.
 4. **Q: Which end of the stack does push operate on?** A: The top of the stack — the only end where insertion is allowed.
-

10. Pop

1. Concept

- **What it does:** Removes and returns the element from the **top** of the stack.
- **Why it is needed:** It is the only way to remove data from a stack, always maintaining the LIFO order.
- **When it is used:** Undo operations, backtracking, returning from a function call, evaluating expressions.

Case	Time Complexity	Why
Best	$O(1)$	Direct removal from the top — no shifting needed
Average	$O(1)$	Same — always a direct operation
Worst	$O(1)$	Same — pop never depends on stack size

2. Algorithm

```
ALGORITHM: PopElement
INPUT: StackArray, TopPointer
```

1. Start
2. If `TopPointer` equals `-1`, then display "Stack Underflow" and Stop
3. Set `RemovedValue` to the element at `TopPointer` in `StackArray`
4. Decrease `TopPointer` by 1
5. Display `RemovedValue`
6. Stop

3. Step-by-Step Explanation

1. **Start** the process.
2. We first check if the stack is already empty (this condition is called **Stack Underflow**). If so, there is nothing to remove, and we stop.
3. We read (save) the value that is currently at the top position, since we need to return it.
4. We decrease the `TopPointer` by one — this logically removes the top element by shrinking the stack's boundary.
5. We display or return the value we removed.
6. **Stop**.

4. Dry Run

StackArray: `[10, 20, 30]`, `TopPointer` = 2

Step	Action	TopPointer	Stack State
Initial	—	2	<code>[10, 20, 30]</code>
Check Underflow	<code>TopPointer</code> (2) $\neq$ -1 $\rightarrow$ proceed	2	<code>[10, 20, 30]</code>
Save Value	<code>RemovedValue</code> = 30	2	<code>[10, 20, 30]</code>
Decrease TopPointer	<code>TopPointer</code> becomes 1	1	<code>[10, 20]</code> (logically)

Popped Value: 30 , **Final Stack:** `[10, 20]` , **TopPointer** = 1

5. Memory Trick

"Check Empty $\rightarrow$ Grab the Top $\rightarrow$ Step Down"

6. Common Mistakes

- Decreasing `TopPointer` **before** reading the value, causing the wrong (or invalid) value to be returned.
- Forgetting to check for Stack Underflow, causing errors when popping an empty stack.

- Assuming the popped value is automatically erased from memory — the value often still physically remains until overwritten; only `TopPointer` logically excludes it.
- Confusing pop with peek (pop removes the element; peek only views it).

7. Viva Questions

1. **Q: What does "Stack Underflow" mean?** A: It means an attempt was made to pop an element from a stack that is already empty.
 2. **Q: Why is pop an $O(1)$ operation?** A: Because it only involves reading a value and decrementing a pointer — no traversal or shifting is needed.
 3. **Q: What is the difference between pop and peek?** A: Pop removes the top element and returns it; peek only returns the top element without removing it.
 4. **Q: Which principle does pop follow?** A: LIFO (Last In, First Out) — the most recently pushed element is always the first one removed.
-

11. Peek

1. Concept

- **What it does:** Returns (views) the element at the **top** of the stack **without removing it**.
- **Why it is needed:** Sometimes we only need to check the current top value without disturbing the stack's contents.
- **When it is used:** Checking the next operation in an undo stack, checking matching brackets, viewing the top of a browser's back-navigation stack.

Case	Time Complexity	Why
Best	$O(1)$	Direct access to the top position
Average	$O(1)$	Same — always a direct operation
Worst	$O(1)$	Same — peek never depends on stack size

2. Algorithm

```
ALGORITHM: PeekElement
INPUT: StackArray, TopPointer
1. Start
2. If TopPointer equals -1, then display "Stack is Empty" and Stop
3. Set TopValue to the element at TopPointer in StackArray
```

4. Display TopValue
5. Stop

3. Step-by-Step Explanation

1. **Start** the process.
2. We check if the stack is empty. If it is, there's no top element to view, so we stop.
3. We read the value stored at the current top position — we do **not** change TopPointer here.
4. We display this value to the user.
5. **Stop** — notice the stack remains completely unchanged.

4. Dry Run

StackArray: [10, 20, 30], TopPointer = 2

Step	Action	TopPointer	Stack State
Initial	—	2	[10, 20, 30]
Check Empty	TopPointer (2) $\neq$ -1 $\rightarrow$ proceed	2	[10, 20, 30]
Read Value	TopValue = 30	2	[10, 20, 30] (unchanged)

Peeked Value: 30, **Stack remains:** [10, 20, 30]

5. Memory Trick

"Check Empty $\rightarrow$ Look, Don't Touch"

6. Common Mistakes

- Accidentally decreasing TopPointer (this turns peek into pop by mistake).
- Forgetting to check whether the stack is empty before peeking.
- Confusing peek's return value with the **entire stack** rather than just the top single element.
- Assuming peek modifies the stack — it never does.

7. Viva Questions

1. **Q: How is peek different from pop?** A: Peek only views the top element; pop views **and removes** it.
2. **Q: Does peek change the value of TopPointer?** A: No, TopPointer remains exactly the same after a peek operation.

3. **Q: What is the time complexity of peek, and why?** A: $O(1)$, because it directly accesses a single known position (`TopPointer`) without any loop.
4. **Q: Why is peek useful in expression evaluation?** A: It allows checking the current top operator or value without disturbing the evaluation stack's state.
-

12. Display

1. Concept

- **What it does:** Shows all elements currently present in the stack, typically from the top to the bottom.
- **Why it is needed:** Useful for debugging, verifying stack contents, or presenting the current state to a user.
- **When it is used:** Printing the stack for testing, showing undo history, visualizing the current call stack.

Case	Time Complexity	Why
Best	$O(N)$	Every element currently in the stack must be visited
Average	$O(N)$	Same reasoning
Worst	$O(N)$	Same — proportional to the number of elements present

2. Algorithm

```
ALGORITHM: DisplayStack
INPUT: StackArray, TopPointer
1. Start
2. If TopPointer equals -1, then display "Stack is Empty" and Stop
3. Set CurrentIndex to TopPointer
4. Repeat steps 5 to 6 while CurrentIndex is greater than or equal to 0
5.     Display the element at CurrentIndex in StackArray
6.     Decrease CurrentIndex by 1
7. End Repeat
8. Stop
```

3. Step-by-Step Explanation

1. **Start** the process.
2. We check whether the stack has any elements at all; if not, we simply report that it is empty.

3. We begin from the **top** position, since displaying is usually done from most-recently-added to least-recently-added.
4. We continue **as long as** our current position hasn't gone below the very first index (0).
5. We display (print) the value at the current position.
6. We move one step down, toward the bottom of the stack.
7. This loop continues until every element has been shown.
8. **Stop.**

4. Dry Run

StackArray: [10, 20, 30], TopPointer = 2

CurrentIndex	Condition (Index $\geq$ 0)	Element Displayed
2	True	30
1	True	20
0	True	10
-1	False $\rightarrow$ Loop Ends	—

Output: 30 20 10 (top to bottom)

5. Memory Trick

"Check Empty $\rightarrow$ Start at Top $\rightarrow$ Print $\rightarrow$ Step Down $\rightarrow$ Repeat"

6. Common Mistakes

- Starting the display from index 0 instead of TopPointer (this shows bottom-to-top instead of the conventional top-to-bottom).
- Forgetting to check for an empty stack first.
- Using $>$ instead of $\geq$ in the loop condition, which skips displaying the bottom-most element (index 0).
- Confusing TopPointer with NumberOfElements when they are tracked differently.

7. Viva Questions

1. **Q: In what order does display typically show stack elements?** A: From the top element to the bottom element, reflecting LIFO order.
2. **Q: What is the time complexity of display, and why?** A: $O(N)$, since every currently stored element must be visited and printed once.

3. **Q: What happens if display is called on an empty stack?** A: It should print a message such as "Stack is Empty" instead of attempting to loop.
4. **Q: Does display modify the stack in any way?** A: No, it only reads and shows values; `TopPointer` remains unchanged.
-

13. isEmpty

1. Concept

- **What it does:** Checks whether the stack currently has **zero elements**.
- **Why it is needed:** Prevents invalid operations like popping or peeking from an empty stack (Stack Underflow).
- **When it is used:** As a safety check before every pop or peek operation, or to control loops that process a stack until it's empty.

Case	Time Complexity	Why
Best	$O(1)$	A single comparison of <code>TopPointer</code>
Average	$O(1)$	Same — always a direct comparison
Worst	$O(1)$	Same — never depends on stack size

2. Algorithm

```
ALGORITHM: IsStackEmpty
INPUT: TopPointer
1. Start
2. If TopPointer equals -1, then return True
3. Otherwise, return False
4. Stop
```

3. Step-by-Step Explanation

1. **Start** the process.
2. We compare `TopPointer` with `-1`, the value that conventionally represents "no elements present."
3. If they are equal, the stack has nothing in it, so we return `True`.
4. If `TopPointer` is anything other than `-1`, at least one element exists, so we return `False`.
5. **Stop**.

4. Dry Run

Case A: `TopPointer = -1`

- Comparison: `-1 == -1` → **True**
- **Result:** Stack is empty.

Case B: `TopPointer = 2`

- Comparison: `2 == -1` → **False**
- **Result:** Stack is not empty.

5. Memory Trick

"**TopPointer at Minus One? → Nothing's Home**"

6. Common Mistakes

- Comparing `TopPointer` with `0` instead of `-1` (mixing up different stack initialization conventions).
- Forgetting to call `isEmpty` before performing `pop` or `peek`, leading to underflow errors.
- Returning the wrong Boolean value (mixing up `True` and `False`).
- Confusing `isEmpty` with `isFull` — they check opposite conditions.

7. Viva Questions

1. **Q: Why is `isEmpty` important before calling `pop`?** A: To avoid a **Stack Underflow** error, which happens when trying to remove from an already-empty stack.
 2. **Q: What does `TopPointer` equal to `-1` signify?** A: That the stack currently contains no elements.
 3. **Q: What is the time complexity of `isEmpty`?** A: $O(1)$, since it only involves one simple comparison.
 4. **Q: Can `isEmpty` and `isFull` both return `True` at the same time?** A: No — unless the stack's maximum size is 0, these two conditions are mutually exclusive.
-

14. isFull

1. Concept

- **What it does:** Checks whether the stack has reached its **maximum capacity** and cannot accept any more elements.

- **Why it is needed:** Prevents invalid operations like pushing onto a full stack (Stack Overflow).
- **When it is used:** As a safety check before every push operation, especially in fixed-size (array-based) stack implementations.

Case	Time Complexity	Why
Best	$O(1)$	A single comparison of <code>TopPointer</code> with <code>MaximumSize</code>
Average	$O(1)$	Same — always a direct comparison
Worst	$O(1)$	Same — never depends on stack size

2. Algorithm

```

ALGORITHM: IsStackFull
INPUT: TopPointer, MaximumSize
1. Start
2. If TopPointer equals (MaximumSize - 1), then return True
3. Otherwise, return False
4. Stop

```

3. Step-by-Step Explanation

1. **Start** the process.
2. We compare `TopPointer` with `MaximumSize - 1`, which represents the **last valid index** in the stack array.
3. If `TopPointer` has reached this last valid index, the stack has no more room, so we return `True`.
4. If `TopPointer` is smaller than this value, there is still space available, so we return `False`.
5. **Stop**.

4. Dry Run

Case A: `MaximumSize = 5`, `TopPointer = 4`

- Comparison: $4 == (5 - 1) \rightarrow 4 == 4 \rightarrow \text{True}$
- **Result:** Stack is full.

Case B: `MaximumSize = 5`, `TopPointer = 2`

- Comparison: $2 == (5 - 1) \rightarrow 2 == 4 \rightarrow \text{False}$
- **Result:** Stack is not full.

5. Memory Trick

"TopPointer Hits the Last Seat? → No Room Left"

6. Common Mistakes

- Comparing `TopPointer` with `MaximumSize` instead of `MaximumSize - 1` (off-by-one error, since indexing starts at 0).
- Forgetting to call `isFull` before performing push, leading to overflow errors.
- Confusing `isFull` with `isEmpty` — they check opposite ends of the stack's capacity.
- Hardcoding the maximum size instead of using the actual declared capacity variable.

7. Viva Questions

1. **Q: Why is `isFull` important before calling `push`?** A: To avoid a **Stack Overflow** error, which happens when trying to add to an already-full stack.
2. **Q: What does `TopPointer` equal to (`MaximumSize - 1`) signify?** A: That every available slot in the stack array is currently occupied.
3. **Q: What is the time complexity of `isFull`?** A: $O(1)$, since it only involves one simple comparison.
4. **Q: Is `isFull` relevant for all stack implementations?** A: It is mainly relevant for **array-based (fixed-size)** stacks; dynamic or linked-list-based stacks generally don't have a fixed capacity, so overflow is rarely a concern there.

Quick-Glance Summary Table

Operation	Data Structure	Best Case	Average Case	Worst Case
Traversal	Array	$O(N)$	$O(N)$	$O(N)$
Insertion	Array	$O(1)$	$O(N)$	$O(N)$
Deletion	Array	$O(1)$	$O(N)$	$O(N)$
Linear Search	Array	$O(1)$	$O(N)$	$O(N)$
Binary Search	Array	$O(1)$	$O(\log N)$	$O(\log N)$
Updating	Array	$O(1)$	$O(1)$	$O(1)$
Merging	Array	$O(N+M)$	$O(N+M)$	$O(N+M)$
Reversing	Array	$O(N)$	$O(N)$	$O(N)$
Push	Stack	$O(1)$	$O(1)$	$O(1)$

Operation	Data Structure	Best Case	Average Case	Worst Case
Pop	Stack	$O(1)$	$O(1)$	$O(1)$
Peek	Stack	$O(1)$	$O(1)$	$O(1)$
Display	Stack	$O(N)$	$O(N)$	$O(N)$
isEmpty	Stack	$O(1)$	$O(1)$	$O(1)$
isFull	Stack	$O(1)$	$O(1)$	$O(1)$

Final exam tip: If you remember nothing else, remember this — **Arrays are fast to read ($O(1)$ update/access) but slow to insert/delete in the middle ($O(N)$). Stacks are fast at everything ($O(1)$) because they only ever touch the top.**

Good luck for your exam! 🎯